virvelle

CORSO DI MOBILE PROGRAMMING

PROGRAMMA GOL

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary

Finanziato dall'Unione europea
NextGenerationEU

Benvenuti al Laboratorio 5! 💻

Fino ad oggi, ogni volta che chiudevamo la nostra app, tutti i dati sparivano nel nulla. Era come scrivere sulla sabbia.
Oggi impariamo a scrivere sulla roccia: i dati sopravviveranno alla chiusura dell'app, al riavvio del telefono e persino agli aggiornamenti.

Come funzionano i Laboratori:

1. 🧠 **Briefing & Recap:** 10 minuti di ripasso.
2. 👨‍🏫 **Scaffolding (Insieme):** Scriviamo l'architettura base guidati da me.
3. 🚀 **Sfida Autonoma:** Aggiungete funzionalità da soli (o in team).

Recap Rapido: I 3 Mattoncini di Oggi

Per rendere i dati persistenti, useremo la libreria **Room**:

1. **L'Entity (`@Entity`):** 💡 *È la nostra tabella del database. Se `remember` è un post-it che buttiamo via, l'Entity è una riga scritta in un registro contabile permanente.*
2. **Il DAO (`@Dao`):** 💡 *È l'unico che ha le chiavi del magazzino. È un'interfaccia dove definiamo le azioni: "Inserisci pasto", "Cancella pasto", "Dammi tutta la lista".*
3. **AppDatabase:** 💡 *È il magazzino fisico. È un oggetto pesante che creiamo una volta sola (Singleton) per gestire la connessione sicura ai nostri dati senza bloccare l'app.*

virvelle

Il Progetto di Oggi

🍎 **Diario Alimentare (Persistenza Room)**

Il Problema Reale

Un utente che usa un'app per la dieta vuole poter controllare cosa ha mangiato ieri o una settimana fa.
Non possiamo affidarci alla memoria RAM: dobbiamo salvare ogni pasto in un database locale (SQLite) in modo che sia sempre disponibile, anche senza connessione internet.

Architettura dell'App

Costruiremo un diario dove annotare i pasti della giornata:

- **Il Modello:** Ogni pasto avrà un nome (es. "Pasta al forno"), una categoria (es. "Pranzo") e magari le calorie.
- **La Lista:** Useremo una `LazyColumn` che legge i dati direttamente dal Database in tempo reale.
- **L'Inserimento:** Un modulo semplice per aggiungere nuovi pasti al nostro storico.

virvelle

Fase 1: Scaffolding

(Lo facciamo insieme)

Cosa faremo ora nella Fase 1

Condividerò lo schermo e configureremo il "motore" del database. È la parte più tecnica, occhio ai dettagli!

1. Creeremo la `data class Pasto` con l'annotazione `@Entity`.
2. Scriveremo il `PastoDao` con le funzioni per inserire e leggere i dati.
3. Configureremo la classe `AppDatabase` (il Singleton).
4. Creeremo la funzione `inserisciPasto` nel ViewModel usando una **Coroutine** (`viewModelScope.launch`).

Room è molto rigido: basta un errore in una virgola e l'app non compila. Pronti?

virvelle

Fase 2: Sfida Autonoma

(Tocca a voi)

Il vostro turno

Il magazzino è pronto, ma l'app è ancora grezza. Dobbiamo permettere all'utente di gestire i propri pasti in modo moderno e sicuro.

Avete **tre Task** da completare per finire il laboratorio: Grafica, Logica interattiva (Swipe) e il classico Crash Test di Room.

🎨 Task 1: Sfida Grafica (Categorie & Icone)

Vedere solo una lista di nomi è noioso. Rendiamo il diario visivamente utile.

- **Obiettivo:** Categorizzare i pasti.
- Aggiungete un campo "Categoria" (es. Colazione, Pranzo, Cena, Snack) all'Entity.
- Nella riga della lista, mostrate un'icona o un colore diverso in base alla categoria.
- Usate una `Card` con un design pulito per ogni pasto inserito.

⚙️ Task 2: Sfida Logica (Swipe to Delete)

Cancellare un elemento premendo un tastino è superato. Vogliamo un'interazione moderna.

- **Obiettivo:** Implementare lo **"Swipe to Delete"**.
- Fate in modo che l'utente possa trascinare una riga verso sinistra per cancellare il pasto dal database.
- **Suggerimento:** Cercate come usare il componente `SwipeToDismiss` (o `SwipeToDismissBox`) di Jetpack Compose. Dovrete collegare l'azione di "Dismiss" alla funzione `delete` del vostro DAO.

💥 Task 3: Crash Test (Main Thread Violation)

Room ha una protezione di sicurezza integrata: vi impedisce di fare operazioni pesanti sul thread della UI.

Obiettivo: Vedere il crash "Database on Main Thread".

- Togliete temporaneamente il `viewModelScope.launch` dalla funzione che inserisce il pasto, chiamando il DAO direttamente.
- Eseguite l'app e provate a salvare.
- **Il risultato:** L'app esploderà. Leggete il messaggio nel **Logcat**: imparerete a riconoscere l'errore `java.lang.IllegalStateException: Cannot access database on the main thread`.
- *Rimettete a posto il codice dopo il test!*

virvelle

🤖 Un avviso sull'Intelligenza Artificiale

Siete Ingegneri, non Copiatori.

Come usare l'IA nel modo giusto

Lo "Swipe to Delete" è una funzionalità avanzata. L'IA può aiutarvi con la struttura sintattica, ma voi dovete gestire i dati.

❌ **Prompt Sbagliato:** *"Fammi lo swipe to delete per Room."*
(Vi darà codici complessi con stati che non saprete gestire).

✅ **Prompt Corretto (Da Ingegnere):** *"In Jetpack Compose, come posso usare SwipeToDismissBox per eliminare un elemento da una lista? Fammi un esempio semplice della parte UI."*

Il ruolo del Copilota

Se l'IA vi suggerisce di cambiare la "Versione" del Database o di fare una "Migration", fate attenzione! Se modificate l'Entity (Task 1), la via più veloce in laboratorio è **disinstallare l'app dall'emulatore** e reinstallarla. Questo cancellerà il vecchio DB e creerà quello nuovo senza errori.

Let's Code! 🚀

Condivido lo schermo. Cominciamo definendo la nostra prima Entity: il Pasto!

virvelle

virvelle

Learn Your Way, Grow With Us.

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary